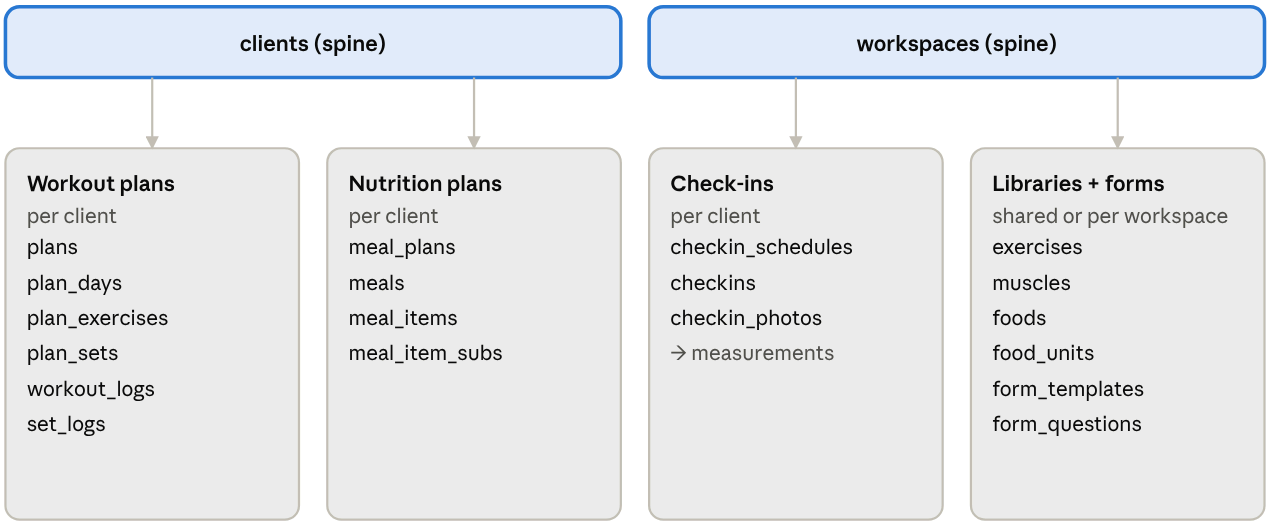
Database Schema II — Everything Past the Spine

The form builder, workout plans, nutrition, check-ins and platform billing. Every table here is additive: nothing in the spine changes, which is what made deferring them safe.

Overview

What attaches to what

The spine stays unchanged; each cluster hangs off clients or workspaces



Billing sits apart: payments (spine) already covers client money; platform_subscriptions is its own concern.

new clusters · attachment points

Two structural rules run through everything below.

Templates are copies, never references. A plan built from a template, a meal item built from a food, a check-in built from a schedule — each copies the values it needs at creation time. This was the decision in the spine for `subscriptions.package_name`, and it applies identically here: editing a template must never rewrite a client's history.

Libraries are shared, overrides are per workspace. `exercises` and `foods` are global rows you seed and maintain. A coach who adds his own exercise writes a row in the same table with a `workspace_id` set; global rows have `workspace_id` null. One table, one query, two visibility rules.

Form builder

The spine shipped one hard-coded form. This replaces it without invalidating anything already submitted — which is what `intake_responses.form_version` was reserving space for.

```
create table form_templates (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid references workspaces(id) on delete cascade,  -- null =
global starter
  name          text not null,
  purpose       text not null default 'intake'
                  check (purpose in ('intake','checkin')),
  version       int not null default 1,
  is_active     boolean not null default true,
  created_at    timestamptz not null default now()
);

create table form_questions (
  id            uuid primary key default gen_random_uuid(),
  template_id   uuid not null references form_templates(id) on delete
cascade,
  position      int not null,
  label_ar      text not null,
  label_en      text,
  help_text     text,
  input_type    text not null check (input_type in (

'short_text','long_text','number','single_select','multi_select',
                  'dropdown','date','image','boolean','scale')),
  options       jsonb,                          -- for select types:
[{value,label_ar}]
  is_required   boolean not null default false,
  unit          text,                           -- 'kg', 'cm' for number
inputs
  maps_to_metric text,                          -- 'weight' -> writes into
measurements
  created_at    timestamptz not null default now(),
  unique (template_id, position)
);
```

`maps_to_metric` is the important column

It's what keeps the charts working once coaches write their own questions. A number question tagged `maps_to_metric = 'weight'` gets copied into `measurements` on submission; an untagged one just lives in the jsonb. Without it, a coach who rewrites his intake form silently breaks his own progress charts.

Versioning without a version table

When a coach edits a published template, don't mutate it. Insert a new row with `version + 1` and flip `is_active`. Responses point at a specific template id, so an old response stays interpretable forever.

```
-- intake_responses gains one column
alter table intake_responses
  add column template_id uuid references form_templates(id);
```

Keep `form_version` text alongside it for the rows written before this table existed. Backfill it to the global starter template's id once, then treat the text column as legacy.

Starter templates

Seed the default intake form as a `form_templates` row with `workspace_id = null`. When a coach wants to customize, you copy the global template and its questions into his workspace rather than letting him edit the shared row:

```
create function fork_template(p_template uuid, p_workspace uuid)
returns uuid language plpgsql security definer as $$
declare v_new uuid;
begin
  insert into form_templates (workspace_id, name, purpose, version)
    select p_workspace, name, purpose, 1 from form_templates where id =
p_template
    returning id into v_new;

  insert into form_questions (template_id, position, label_ar, label_en,
help_text,
                              input_type, options, is_required, unit,
maps_to_metric)
    select v_new, position, label_ar, label_en, help_text,
           input_type, options, is_required, unit, maps_to_metric
    from form_questions where template_id = p_template;

  return v_new;
end $$;
```

This fork pattern repeats three more times below — for plan templates, meal plan templates and check-in schedules. Write it once and generalise.

Exercise library

```
create table muscles (
  id            text primary key,              -- 'chest', 'lats', 'quads'
  label_ar      text not null,
  label_en      text not null,
  svg_path_id   text not null,                 -- the path id in the anatomy
SVG
  body_side     text not null check (body_side in ('front','back','both'))
);

create table exercises (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid references workspaces(id) on delete cascade,  -- null =
global
  name_ar       text not null,
  name_en       text not null,
  slug          text,
```

```
  equipment     text[] not null default '{}',  -- {'barbell','dumbbell'}
  difficulty    text not null default 'beginner'
                  check (difficulty in
('beginner','intermediate','advanced')),
  cues          text[] not null default '{}',  -- 3 short lines
  mistakes      text[] not null default '{}',  -- 2 lines
  video_url     text,                          -- a YouTube link, seeded or
coach-supplied
  is_active     boolean not null default true,
  created_at    timestamptz not null default now()
);

create index on exercises (workspace_id);
create index on exercises using gin (equipment);
create index on exercises using gin (
  to_tsvector('simple', name_ar || ' ' || name_en)
);

create table exercise_muscles (
  exercise_id   uuid not null references exercises(id) on delete cascade,
  muscle_id     text not null references muscles(id) on delete restrict,
  role          text not null check (role in ('primary','secondary')),
  primary key (exercise_id, muscle_id)
);
```

Why `muscles` is a real table with a text PK

Unlike `measurements.metric`, the muscle list genuinely is closed — a coach does not invent new muscles. A foreign key here is correct, and the text primary key (`'chest'`, not a uuid) means your SVG rendering code reads naturally and your seed data is human-readable.

`svg_path_id` is the join between data and drawing. Your anatomy component loads the two SVGs once, queries `exercise_muscles` for the exercise, and fills the matching paths. One component, every exercise.

Global and workspace rows in one table

`workspace_id null` means a library exercise everyone sees; a uuid means a coach's private addition. The visibility rule is one clause:

```
-- what this coach can see
select * from exercises
where is_active
  and (workspace_id is null or workspace_id in (select
auth_workspace_ids()));
```

The alternative — separate `exercises` and `custom_exercises` tables — means every query that touches exercises becomes a union, forever. Don't.

The equipment array

`text[]` with a GIN index rather than a join table. Equipment is a short, closed, read-mostly list that's only ever filtered with "contains any of", which is exactly what `&&` on a GIN-indexed array does well:

```
-- exercises a home client can actually do
select * from exercises
where equipment && array['bodyweight','bands'];
```

A join table would be more normalised and buy you nothing here.

Deactivation, never deletion

`is_active = false` because plans reference exercises. This was in the edge-case list and the foreign key enforces it anyway — `on delete restrict` would block a delete that plans depend on, but marking inactive is the path you actually want: it disappears from search while old plans keep rendering.

Workout plans

```
create table plans (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid references clients(id) on delete cascade,  -- null = a
template
  is_template   boolean not null default false,
  name          text not null,
  goal          text,
  weeks         int not null default 4 check (weeks between 1 and 52),
  status        text not null default 'draft'
                  check (status in ('draft','active','ended')),
```

```
  starts_on     date,
  notes         text,
  created_by    uuid references coaches(id) on delete set null,
  created_at    timestamptz not null default now(),
  check ((is_template and client_id is null) or (not is_template and
client_id is not null))
);

-- one active plan per client
create unique index one_active_plan_per_client
  on plans (client_id) where status = 'active' and client_id is not null;

create table plan_days (
  id            uuid primary key default gen_random_uuid(),
  plan_id       uuid not null references plans(id) on delete cascade,
  week_no       int not null check (week_no > 0),
  day_no        int not null check (day_no between 1 and 7),
  name          text,                          -- 'صدر وتراي'
  is_rest       boolean not null default false,
  is_deload     boolean not null default false,
  notes         text,
  unique (plan_id, week_no, day_no)
);

create table plan_exercises (
  id            uuid primary key default gen_random_uuid(),
  plan_day_id   uuid not null references plan_days(id) on delete cascade,
  exercise_id   uuid not null references exercises(id) on delete restrict,
  position      int not null,
  exercise_name text not null,                 -- snapshot
  coach_note    text,
  superset_group text,                         -- same letter = same superset
  unique (plan_day_id, position)
);

create table plan_sets (
  id              uuid primary key default gen_random_uuid(),
  plan_exercise_id uuid not null references plan_exercises(id) on delete
cascade,
  set_no          int not null,
  reps            text,                        -- '8-10' or '12' or 'AMRAP'
  weight_kg       numeric(6,2),
  weight_pct      numeric(5,2),                -- % of 1RM, alternative to
```

```
absolute
  rest_seconds    int,
  rir             numeric(3,1) check (rir >= 0),  -- reps in reserve
  unique (plan_exercise_id, set_no)
);
```

Templates live in the same table

`is_template = true, client_id = null` rather than a separate `plan_templates` table. The check constraint keeps the two states honest, and building a client plan from a template is a deep copy down four levels — the same fork pattern as forms.

The payoff: one set of queries, one editor UI, and "save this plan as a template" is an insert, not a conversion between two shapes.

`reps` is text, deliberately

Coaches write `8-10`, `AMRAP`, `12 each side`, `30 sec`. An integer column forces you to either reject real programming or invent four more columns. This is display-and-copy data, not something you compute with — text is the honest type.

`weight_kg` and `weight_pct` are separate and both nullable: some programming is absolute, some is percentage-based, and a beginner plan has neither.

Client logging

```
create table workout_logs (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  plan_day_id   uuid references plan_days(id) on delete set null,
  performed_on  date not null,
  status        text not null default 'completed'
                  check (status in ('completed','partial','skipped')),
  duration_min  int,
  client_note   text,
  created_at    timestamptz not null default now(),
  unique (client_id, plan_day_id, performed_on)
);

create table set_logs (
  id              uuid primary key default gen_random_uuid(),
```

```
  workout_log_id  uuid not null references workout_logs(id) on delete
cascade,
  plan_set_id     uuid references plan_sets(id) on delete set null,
  exercise_id     uuid not null references exercises(id) on delete restrict,
  set_no          int not null,
  reps_done       int,
  weight_kg       numeric(6,2),
  rir             numeric(3,1) check (rir >= 0)
);

create index on workout_logs (client_id, performed_on desc);
create index on set_logs (exercise_id);
```

`plan_day_id` and `plan_set_id` are both nullable with `on delete set null`, because a client may log a workout that isn't in any plan, and because deleting a plan must never delete the client's training history.

That index on `set_logs (exercise_id)` is what powers "show me your bench press over the last six months" — the single most requested client feature once people start logging.

Food database

This is the asset no global competitor has. It's also the one place where getting the units wrong makes the whole feature unusable.

```
create table foods (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid references workspaces(id) on delete cascade,  -- null =
global
  name_ar       text not null,
  name_en       text,
  category      text not null check (category in (
                  'protein','carb','fat','vegetable','fruit','dairy',
                  'drink','composite','supplement')),
  -- macros are ALWAYS per 100 g, no exceptions
  kcal_100g     numeric(7,2) not null,
  protein_100g  numeric(6,2) not null,
  carb_100g     numeric(6,2) not null,
  fat_100g      numeric(6,2) not null,
  fiber_100g    numeric(6,2) not null default 0,
  is_active     boolean not null default true,
  source        text,                          -- 'usda', 'nni_egypt',
```

```
'manual'
  created_at    timestamptz not null default now()
);

create index on foods (workspace_id);
create index on foods using gin (to_tsvector('simple', name_ar));

create table food_units (
  id            uuid primary key default gen_random_uuid(),
  food_id       uuid not null references foods(id) on delete cascade,
  label_ar      text not null,                -- 'قطعة', 'كوب', 'رغيف'
متوسطة'
  grams         numeric(7,2) not null check (grams > 0),
  is_default    boolean not null default false,
  position      int not null default 0
);

create unique index one_default_unit_per_food
  on food_units (food_id) where is_default;
```

Store macros per 100 g and nothing else

Every alternative — per serving, per piece, per package — turns macro arithmetic into unit bookkeeping at every call site. One canonical basis plus a conversion table means the math is always `grams / 100 * macro`, everywhere, forever.

`food_units` is then pure translation: `1 رغيف → 90 g`, `1 كوب أرز → 158 g`. This is the table that makes the product usable by someone who will never own a kitchen scale, and it's cheap: most foods need two or three rows.

Composite dishes are rows, not recipes

Koshari is one `foods` row with `category = 'composite'` and macros for an average portion, plus a `food_units` row of `1 طبق متوسط → 450 g`. It is **not** modelled as a recipe of rice, lentils and pasta.

The reasoning: the coach picks it in one second, the client recognises it, and nobody needs to know the lentil-to-rice ratio. If you later want recipes, that's an additive table that doesn't disturb this one.

```
-- only if you ever need it; not for v2
create table food_components (
  composite_id  uuid not null references foods(id) on delete cascade,
  component_id  uuid not null references foods(id) on delete restrict,
  grams         numeric(7,2) not null,
  primary key (composite_id, component_id)
);
```

Seeding strategy

Source	What you take	Rows
USDA FoodData Central	Raw ingredients: meats, grains, vegetables, dairy	~200
Egyptian National Nutrition Institute tables	Local staples and their composition	~60
Manual calculation	Composite dishes built from the above	~40

The 40 manual rows are the ones that matter, and they are a week of careful work by one person, not an engineering task. Tag everything with `source` so you can re-verify later when someone disputes a number — and they will.

Nutrition plans

```
create table meal_plans (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid references clients(id) on delete cascade,
  is_template   boolean not null default false,
  name          text not null,
  kcal_target   int,
  protein_target_g int,
  carb_target_g    int,
  fat_target_g     int,
  status        text not null default 'draft'
                  check (status in ('draft','active','ended')),
  starts_on     date,
  notes         text,
```

```
  created_at    timestamptz not null default now(),
  check ((is_template and client_id is null) or (not is_template and
client_id is not null))
);

create unique index one_active_meal_plan_per_client
  on meal_plans (client_id) where status = 'active' and client_id is not
null;

create table meals (
  id            uuid primary key default gen_random_uuid(),
  meal_plan_id  uuid not null references meal_plans(id) on delete cascade,
  day_no        int not null default 0 check (day_no between 0 and 7),  -- 0
= every day
  position      int not null,
  name          text not null,                -- 'سناك' ,'فطار'
  time_hint     time,
  notes         text,
  unique (meal_plan_id, day_no, position)
);

create table meal_items (
  id            uuid primary key default gen_random_uuid(),
  meal_id       uuid not null references meals(id) on delete cascade,
  food_id       uuid not null references foods(id) on delete restrict,
  position      int not null,

  -- what the coach picked
  quantity      numeric(7,2) not null check (quantity > 0),
  unit_id       uuid references food_units(id) on delete set null,
  grams         numeric(7,2) not null,         -- resolved at save time

  -- snapshot, so edits to foods never rewrite old plans
  food_name     text not null,
  kcal          numeric(7,2) not null,
  protein_g     numeric(6,2) not null,
  carb_g        numeric(6,2) not null,
  fat_g         numeric(6,2) not null,

  unique (meal_id, position)
);

create table meal_item_subs (
```

```
  id            uuid primary key default gen_random_uuid(),
  meal_item_id  uuid not null references meal_items(id) on delete cascade,
  food_id       uuid not null references foods(id) on delete restrict,
  quantity      numeric(7,2) not null,
  unit_id       uuid references food_units(id) on delete set null,
  grams         numeric(7,2) not null,
  food_name     text not null,
  kcal          numeric(7,2) not null,
  protein_g     numeric(6,2) not null,
  carb_g        numeric(6,2) not null,
  fat_g         numeric(6,2) not null
);
```

Per-day meals

`meals.day_no` is what lets one plan hold seven different days. `0` means the meal repeats every day, `1` to `7` pin it to a weekday.

The default of `0` matters: most coaches write one daily plan and repeat it, and forcing them to build seven identical days would be the kind of friction that sends them back to Word. A coach who wants variety switches that meal to a specific day and adds the others.

```
-- the client's plan for Tuesday: pinned meals plus the everyday ones
select * from meals
where meal_plan_id = $1
  and day_no in (0, 2)
order by day_no, position;
```

The same trick applies to `meal_plan_totals` — group by `day_no` as well as plan, so a seven-day plan reports seven daily totals rather than one impossible sum:

```
create or replace view meal_plan_day_totals as
select mp.id as meal_plan_id, mp.workspace_id, m.day_no,
       mp.kcal_target,
       sum(mi.kcal)      as kcal,
       sum(mi.protein_g) as protein_g,
       sum(mi.carb_g)    as carb_g,
       sum(mi.fat_g)     as fat_g,
       mp.kcal_target - sum(mi.kcal) as kcal_gap
from meal_plans mp
join meals m       on m.meal_plan_id = mp.id
join meal_items mi on mi.meal_id = m.id
group by mp.id, m.day_no;
```

Resolve grams and macros at save time

`meal_items` stores both what the coach chose (`quantity` + `unit_id`) and what it resolved to (`grams` plus the four macro columns). That looks redundant and isn't:

- The **choice** is what you show back in the editor — "2 أرغفة", not "180 g".
- The **resolution** is what you sum. Totals become `sum(kcal)` over the meal, with no joins to `foods` and `food_units` on every page load.
- The **snapshot** means correcting a food's macros next month doesn't silently change a plan a client is already following.

The cost is that fixing a wrong macro value doesn't propagate to existing plans. That's the right trade — a plan a coach reviewed and sent is a document, not a live query.

Daily totals

```
create view meal_plan_totals as
select mp.id as meal_plan_id,
       mp.workspace_id,
       mp.kcal_target,
       sum(mi.kcal)      as kcal,
       sum(mi.protein_g) as protein_g,
       sum(mi.carb_g)    as carb_g,
       sum(mi.fat_g)     as fat_g,
       mp.kcal_target - sum(mi.kcal) as kcal_gap
from meal_plans mp
join meals m       on m.meal_plan_id = mp.id
join meal_items mi on mi.meal_id = m.id
group by mp.id;
```

`kcal_gap` is the "you're 18 g of protein short" indicator from the flows document. Computing it in a view rather than the application means the coach's editor, the client's app and any report all agree on the number.

Substitutes are a flat child table

`meal_item_subs` carries the same snapshot columns as `meal_items`, which is duplication you should accept rather than normalise. A substitute is a complete alternative the client can act on alone — making it a self-referencing row in `meal_items` would require a nullable parent, a check constraint, and recursive queries to render one meal.

Check-ins

```
create table checkin_schedules (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  template_id   uuid references form_templates(id) on delete set null,
  frequency     text not null default 'weekly'
                  check (frequency in
('daily','every_2_days','weekly','fortnightly')),
  day_of_week   int check (day_of_week between 0 and 6),   -- for weekly
  send_at       time not null default '08:00',
  timezone      text not null default 'Africa/Cairo',
  is_active     boolean not null default true,
```

```
  created_at    timestamptz not null default now(),
  unique (client_id)
);

create table checkins (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  schedule_id   uuid references checkin_schedules(id) on delete set null,
  due_on        date not null,
  token         text unique,                  -- same link pattern as intake
  answers       jsonb,
  submitted_at  timestamptz,
  coach_reply   text,
  replied_at    timestamptz,
  replied_by    uuid references coaches(id) on delete set null,
  created_at    timestamptz not null default now(),
  unique (client_id, due_on)
);

create index on checkins (workspace_id, submitted_at desc nulls last);
create index on checkins (client_id, due_on desc);
```

Check-ins are materialised ahead of time

Same reasoning as `reminders` in the spine: generate the next few `checkins` rows with their `due_on` dates rather than computing "is today a check-in day?" on the fly. You get the missed ones for free — a row with `due_on` in the past and `submitted_at` null *is* a missed check-in, which is what powers both the coach's "client went quiet" alert and the adherence percentage.

```
-- clients who have gone quiet: two consecutive misses
select client_id, count(*) as missed
from checkins
where submitted_at is null
  and due_on < current_date
  and due_on > current_date - interval '30 days'
group by client_id
having count(*) >= 2;
```

That query is the at-risk-clients metric from the analytics section. It exists because the rows exist.

Photos get their own table and their own storage

```
create table checkin_photos (
  id            uuid primary key default gen_random_uuid(),
  workspace_id  uuid not null references workspaces(id) on delete cascade,
  client_id     uuid not null references clients(id) on delete cascade,
  checkin_id    uuid references checkins(id) on delete set null,
  storage_key   text not null,                -- private bucket path, never
a public URL
  pose          text check (pose in ('front','side','back','other')),
  taken_on      date not null,
  deleted_at    timestamptz,
  created_at    timestamptz not null default now()
);

create index on checkin_photos (client_id, taken_on desc) where deleted_at is
null;
```

Three rules, restated from the flows document because this is where they become real:

1. **`storage_key`, not `url`.** The row stores a path into a private bucket. Your API mints a signed URL valid for minutes when someone with permission asks. A permanent URL in the database is a permanent leak waiting for one careless screenshot of a database row.
2. **A separate bucket from everything else**, with its own access policy. Exercise videos and logos live in a public bucket; these never share it.
3. **`deleted_at` plus an actual object delete.** The client's right to remove his photos has to erase the file, not just hide the row. Soft-delete the row for referential sanity, then delete the object in the same job.

Don't put photos in `measurements`. They aren't numbers, they have a different lifecycle, and they need a privacy posture nothing else in the schema needs.

Writing numbers through to measurements

On submission, the same `maps_to_metric` mechanism from the form builder applies: tagged numeric answers get upserted into `measurements` with `source = 'checkin'`. The `unique (client_id, metric, measured_at)` constraint from the spine means a re-submitted check-in corrects the point instead of duplicating it.

Billing

No payment gateways, no webhooks, no third-party integration. Money moves outside the system entirely — the client transfers by InstaPay or wallet and sends a screenshot — and the platform's job is to record that it happened.

That means the spine's `payments` table is already the whole story for client-to-coach money: a row per amount received, with `receipt_url` holding the screenshot and `method` saying how it arrived. Nothing here replaces it.

What's left is the other direction: coaches paying you.

Platform billing (coach → you)

```
create table plans_catalog (
  id            text primary key,              -- 'free', 'basic', 'pro'
  name          text not null,
  price_minor   bigint not null,
  currency      char(3) not null default 'EGP',
  client_limit  int,                           -- null = unlimited
  interval      text not null default 'month'
                  check (interval in ('month','year'))
);

create table platform_subscriptions (
  id              uuid primary key default gen_random_uuid(),
  workspace_id    uuid not null references workspaces(id) on delete cascade,
  plan_id         text not null references plans_catalog(id),
  status          text not null default 'trialing'
                    check (status in
('trialing','active','past_due','cancelled')),
  current_period_start date not null,
  current_period_end   date not null,
  cancel_at_period_end boolean not null default false,
  created_at      timestamptz not null default now(),
  unique (workspace_id)
);

create table platform_invoices (
  id              uuid primary key default gen_random_uuid(),
  workspace_id    uuid not null references workspaces(id) on delete cascade,
  amount_minor    bigint not null,
  currency        char(3) not null,
```

```
  period_start    date not null,
  period_end      date not null,
  status          text not null default 'open'
                    check (status in ('open','paid','void','uncollectible')),
  method          text check (method in
('instapay','wallet','cash','bank','other')),
  receipt_url     text,                        -- the coach's transfer
screenshot
  paid_at         timestamptz,
  marked_by       uuid references people(id) on delete set null  -- you,
manually
);

create index on platform_invoices (status, period_end) where status = 'open';
```

Same shape as client payments, same reason: an invoice is marked paid by a human who looked at a screenshot. `marked_by` records which of you did it, which matters the first time two people both think the other confirmed a transfer.

The partial index on open invoices is your collections list — it stays small no matter how many periods accumulate.

Enforcing the client limit

The free tier caps at five clients. Enforce it in the database, not only the UI:

```
create function check_client_limit() returns trigger
language plpgsql as $$
declare v_limit int; v_count int;
begin
  select pc.client_limit into v_limit
  from platform_subscriptions ps
  join plans_catalog pc on pc.id = ps.plan_id
  where ps.workspace_id = new.workspace_id;

  if v_limit is null then return new; end if;

  select count(*) into v_count from clients
  where workspace_id = new.workspace_id and deleted_at is null;

  if v_count >= v_limit then
    raise exception 'client_limit_reached' using errcode = 'P0001';
  end if;
```

```
  return new;
end $$;

create trigger enforce_client_limit
  before insert on clients
  for each row execute function check_client_limit();
```

A limit enforced only in the frontend is not a limit. This also gives you a clean error code to catch and turn into an upgrade prompt.

Why not reuse `subscriptions`

They diverge fast: platform billing runs on periods that roll automatically, trials and a client limit; client subscriptions need freezes, partial cash payments and manual date edits. Sharing a table means every query needs a discriminator, and your revenue reporting has to constantly exclude one kind — which someone will eventually forget to do.

RLS, indexes and build order

Three visibility patterns

Every new table falls into one of three groups. Get the group right and the policy writes itself.

Pattern	Tables	Policy
Tenant-scoped	plans, plan_days*, meal_plans, meals*, checkins, checkin_photos, platform_subscriptions, platform_invoices…	`workspace_id in (select auth_workspace_ids())`
Shared + override	exercises, foods, form_templates	`workspace_id is null or workspace_id in (...)`
Child of a scoped parent	plan_sets, meal_items, form_questions, exercise_muscles, food_units, set_logs	inherit through the parent

```
-- shared + override, read for everyone, write only your own
create policy read_library on exercises
  for select
  using (workspace_id is null or workspace_id in (select
auth_workspace_ids()));

create policy write_own on exercises
  for insert with check (workspace_id in (select auth_workspace_ids()));

create policy update_own on exercises
  for update
  using      (workspace_id in (select auth_workspace_ids()))
  with check (workspace_id in (select auth_workspace_ids()));
```

Splitting read from write is what stops a coach editing the global library. A single `for all` policy would let him.

Deep children: denormalise `workspace_id`

`plan_sets` is three joins from `workspaces`. Writing that as a policy means a correlated subquery on every row read.

```
-- don't do this
create policy p on plan_sets for all using (
  plan_exercise_id in (
    select pe.id from plan_exercises pe
    join plan_days pd on pd.id = pe.plan_day_id
    join plans pl on pl.id = pd.plan_id
    where pl.workspace_id in (select auth_workspace_ids())));
```

Carry `workspace_id` down to every table instead, even the deep ones, and keep it correct with a trigger:

```
alter table plan_days      add column workspace_id uuid not null references
workspaces(id);
alter table plan_exercises add column workspace_id uuid not null references
workspaces(id);
alter table plan_sets      add column workspace_id uuid not null references
workspaces(id);
-- same for meals, meal_items, meal_item_subs, form_questions, set_logs
```

The redundancy is deliberate. It makes every policy identical and every read fast, and the trigger that copies it from the parent on insert is five lines. This is the one place where normalising costs more than it's worth.

Indexes worth adding on day one

```
create index on plans (client_id) where status = 'active';
create index on plan_days (plan_id, week_no, day_no);
create index on meals (meal_plan_id, day_no, position);
create index on meal_items (meal_id, position);
create index on checkins (workspace_id, due_on) where submitted_at is null;
create index on platform_invoices (status) where status = 'open';
```

The partial indexes matter more than the plain ones: your hot queries are all "the unfinished ones", and those stay small forever while the tables grow.

Build order

These are independent of each other, so build them in the order your design partner asks for them — with one dependency worth noting.

#	Cluster	Depends on	Why this order
1	Exercise library	nothing	Seed data work can start immediately and in parallel
2	Workout plans	exercises	The feature coaches ask for first
3	Check-ins	measurements (spine)	Highest retention impact per line of code
4	Form builder	—	Only once coaches complain about the fixed form
5	Food database	nothing	Longest lead time; start the data gathering early
6	Nutrition plans	foods	Don't start before the food data is real
7	Platform billing	—	When you're ready to charge

The one real dependency: **don't build the nutrition plan builder before the food database is genuinely populated.** A meal builder with 40 foods in it demos badly and gets judged as broken, when the problem is seed data. The library work is the long pole — start it now, ship the builder when it's ready.

A note on what this adds up to

The spine was 11 tables. This adds about 22 more. That's a real product and roughly a year of work for a small team — which is the argument for the build order above rather than building it all before launch. Every cluster here is shippable on its own, and the first two coaches will tell you which one to do next more reliably than this table will.